6. Neural audio compression

Neural codecs, CNNs and autoencoders

Key question: can a neural network learn the analysis, quantization and reconstruction steps of an audio codec?

Learning goals

- Describe the encoder–quantizer–decoder structure of neural audio codecs.
- Explain why CNNs are useful for learning audio representations.
- Relate convolution, pooling, bottlenecks and autoencoders to compression.
- Explain residual vector quantization as the discrete coding stage.
- Compare reconstruction, perceptual and adversarial losses.
- Discuss current limits: compute, latency, generalization and evaluation.

From traditional to learned audio coding

Conventional codec

Waveform → fixed analysis transform
→ perceptual model & quantizer
→ entropy coding → bitstream

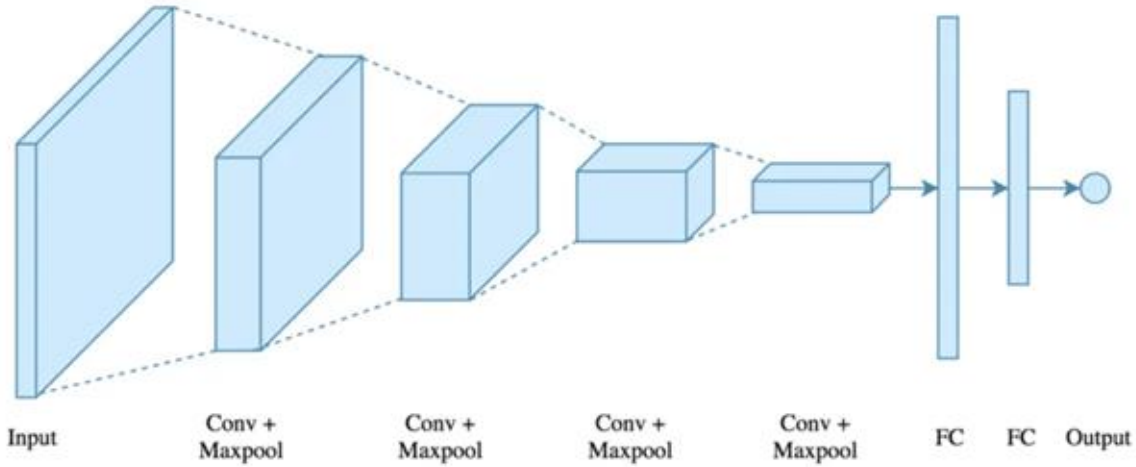
Neural codec

Waveform → learned encoder (CNN)
→ quantized latent tokens
→ coded representation → learned decoder

The signal-processing task is familiar; the analysis/synthesis filters are now learned from examples.

Convolutional Neural Networks (CNNs)

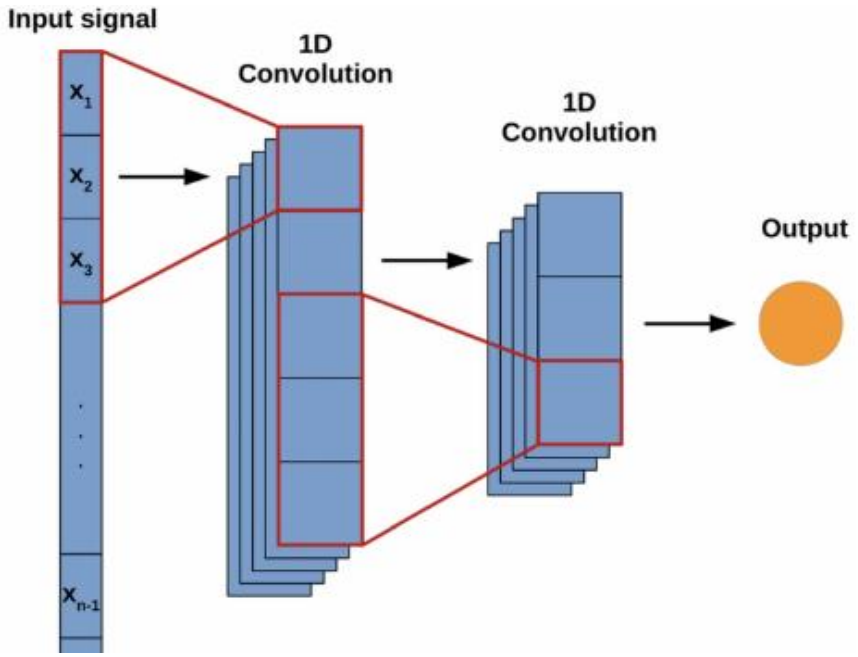
- Originally developed for images but widely applied to audio.
- Learn feature extractors automatically, from low-level to high-level structure.
- For audio, filters may capture transients, timbre, envelopes or local spectral patterns.
- Key strength: automatic feature extraction (from low-level to high-level).



In neural audio coding, the "analysis filter bank" is no longer fixed: it is learned.

What is convolution?

- Mathematical operation between a filter/kernel and the input signal.
- Each position computes a weighted sum over a local neighborhood.
- Unlike classical filters, CNN kernels are learned from training data.



For waveform audio, 1D convolution is a natural way to learn time-local analysis filters.

A 1D convolution: three weighted sums

Input samples	1	2	3	2	1
Learned kernel		1	0	-1	slide kernel →

$$[1, 2, 3] \cdot [1, 0, -1] = -2 \qquad [2, 3, 2] \cdot [1, 0, -1] = 0 \qquad [3, 2, 1] \cdot [1, 0, -1] = +2$$

Resulting feature map: $[-2, 0, +2]$ (stride 1, no padding)

Kernels and feature detection

Learned filters

Small matrices or 1D kernels

Examples: 3×3, 5×5, or short waveform kernels

Audio examples

Low-frequency energy

High-frequency detail

Attacks and transients

Timbre-related local structure

Training

The useful filters are learned from data during optimization.

The model decides which local patterns matter for reconstruction and perceptual quality.

Stride and padding in CNNs

Stride

How much the filter moves between positions.

Larger stride reduces the number of output frames and can contribute to compression.

Padding

Extra samples around the input, often zeros.

Padding controls the output size and avoids losing boundary information.

In learned codecs, strided convolutions often replace explicit downsampling stages.

How a 1D CNN changes the tensor

Input waveform

(1, 1, 4000)

1 mono channel

Conv1d: 4 filters

(1, 4, 500)

stride 8

Decoder

(1, 1, 4000)

ConvTranspose1d

Tensor axes: (batch, channels, time). Four filters produce four feature channels.

Source: convolutional-autoencoder-simple.ipynb · Section 3

Convolutional layer: extracting features

- Each filter produces one feature map.
- Each feature map highlights a specific local pattern.
- Stacked layers capture progressively more complex temporal and spectral structure.

Layer 1: local waveform patterns

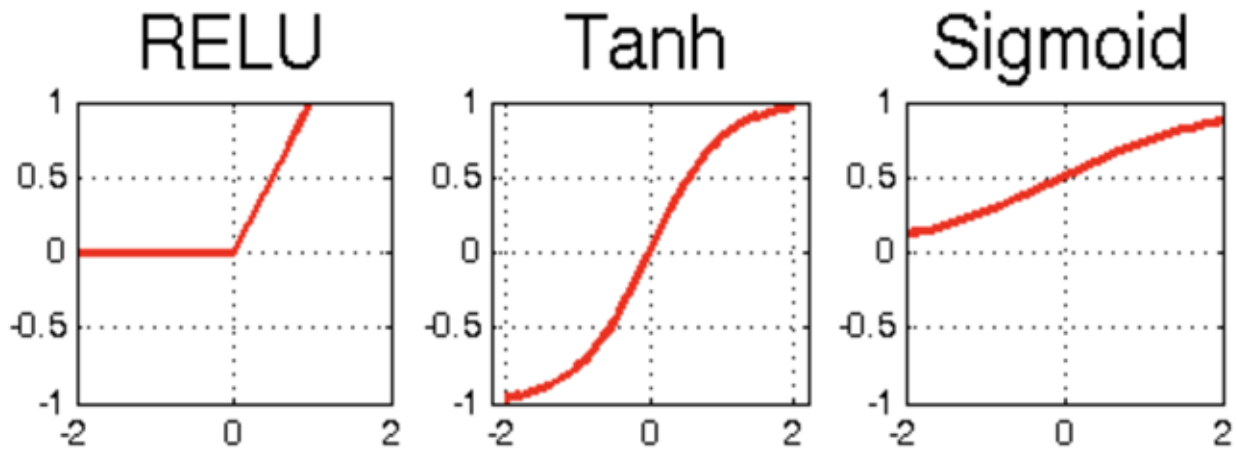
Middle layers: timbre / envelopes

Deep layers: compact latent code

Compression emerges when the latent representation is smaller and quantized.

Non-linear activation functions

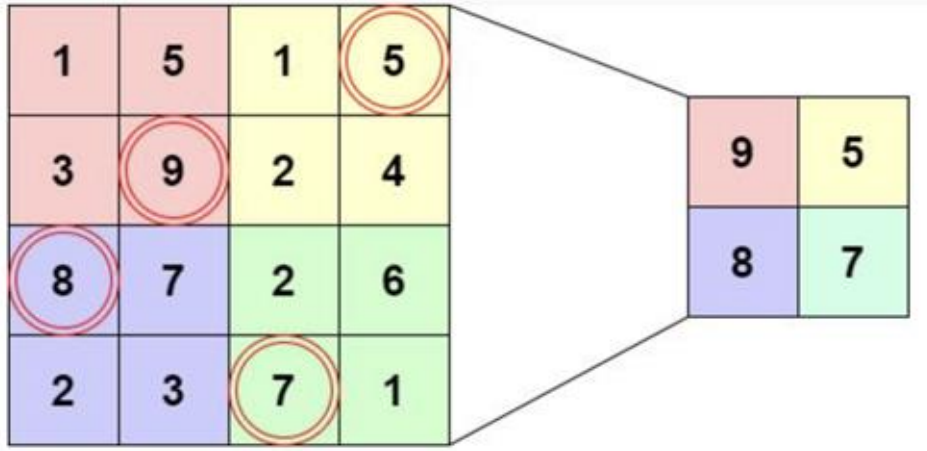
Commonly used: ReLU (Rectified Linear Unit) keeps positive values and sets negative values to zero.



Without non-linearities, a deep network would collapse into a mostly linear transform.

Pooling layer: downsampling the data

- Max-pooling takes the maximum value in a local window.
- It reduces dimensionality and computation.
- It can make the representation less sensitive to small time shifts.



In neural audio codecs, downsampling is often learned with strided convolutions; pooling is optional.

How CNNs build hierarchical representations

Early layers

Local oscillations, attacks, transients and waveform patterns

Deeper layers

Harmonics, timbre, envelopes, longer temporal dependencies

End-to-end training

The model is trained from waveform input to compressed representation and output.

This is the learned counterpart of hand-designed filter banks and transforms.

CNNs vs. traditional audio filters

Traditional audio coding

- Predefined filters and transforms
- Examples: Fourier, MDCT, filter banks
- Designed from signal-processing theory

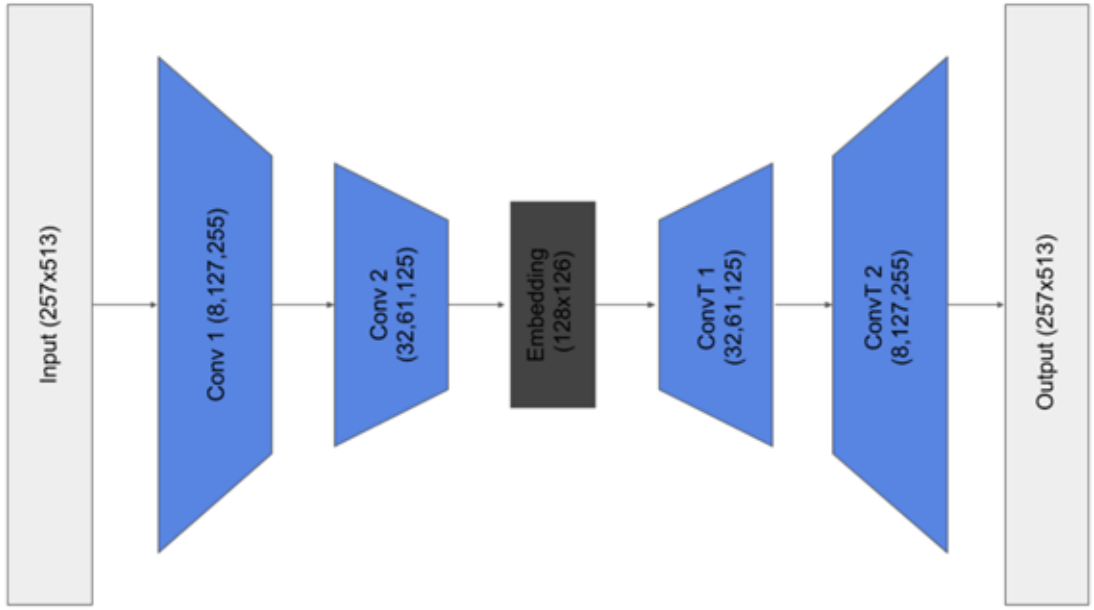
Neural audio coding

- Learned filters and latent spaces
- Optimized for data, loss function and bitrate
- Can adapt to speech, music or mixed audio

The gain depends on whether the learned representation generalizes beyond the training distribution.

Convolutional Autoencoder (CAE)

- Train to reconstruct the input waveform; no audio-content labels are required.
- An encoder learns a compact representation; a decoder reconstructs audio.
- The latent representation must be quantized and coded to form a bitstream.



A smaller continuous latent tensor is not yet a compressed bitstream: quantization and coding are needed.

Example: examples/convolutional-autoencoder-simple.ipynb

How the encoder works in audio compression

- Series of convolutional layers compresses the input audio.
- CNN layers learn features such as harmonics, transients and envelopes.
- The final encoder output is a bottleneck: a compact version of the input.

input waveform → learned analysis → latent representation

How the decoder works in audio compression

- Uses transposed convolutions or related upsampling layers.
- Reconstructs the original signal as accurately and perceptually as possible.
- Success depends on whether the encoder retained the essential audio features.

latent code → learned synthesis → reconstructed waveform

PyTorch: read four lines of code

```
self.analysis = nn.Conv1d(1,4,16,stride=8,padding=4)

z = F.relu(self.analysis(x))

self.synthesis=nn.ConvTranspose1d(4,1,16,stride=8,padding=4)


y = self.synthesis(z)
```

Translate the syntax

1→4: input → feature channels

16: samples per kernel

stride 8: downsampling

padding 4: output alignment

ConvTranspose1d learns synthesis; it is not simply the mathematical inverse of Conv1d.

Source: convolutional-autoencoder-simple.ipynb · network class

Example 1: the simplest possible autoencoder

The fixed network

8 kHz mono input, 0.5 seconds

4,000 waveform samples

Conv1d (1→4; kernel 16, stride 8)

Latent: 4 × 500 = 2,000 floats

Reconstruction

ReLU creates nonlinear features

ConvTranspose1d returns 4,000 samples

Only 133 trainable parameters

No quantizer or entropy coder

Watch the tensor SHAPES before watching the sound quality.

Example 1: what changes when we train?

Version	Training objective	Waveform MSE	Log-spectral distance
A	120 epochs · MSE	0.007689	0.292322
B	500 epochs · MSE + scheduler	0.000251	0.271143
C	500 epochs · MSE + spectral loss	0.000294	0.264009

Compare objectives and trade-offs: C is best on the shown spectral metric, but B has the lowest MSE.

Understanding the bottleneck

Smaller bottleneck

- Fewer latent values
- Potentially lower coded rate
- Higher risk of losing audible detail

Larger bottleneck

- Potentially better reconstruction
- More latent information
- Potentially higher coded rate



The codec design problem is finding the best quality–rate balance for the use case.

Latent size is not yet compressed bitrate

Continuous latent representation

- Example: 4,000 waveform samples → 4 × 500 = 2,000 latent values.
- If both use float32, memory falls from 16,000 to 8,000 bytes.
- This counts tensor values, not a coded file.

Actual neural audio codec

- Quantize latent vectors to discrete indices.
- Pack or entropy-code the indices.
- Account for metadata, padding and decoder requirements.

The bottleneck reduces representation size; quantization and coding determine the transmitted rate.

Training the autoencoder

- Signal loss: often Mean Squared Error (MSE) or waveform/spectral distance.
- Perceptual loss: encourages differences that are less audible.
- Optimization uses backpropagation and gradient descent.

The loss function defines what “good reconstruction” means.

Gradient descent and learning rate

Gradient descent

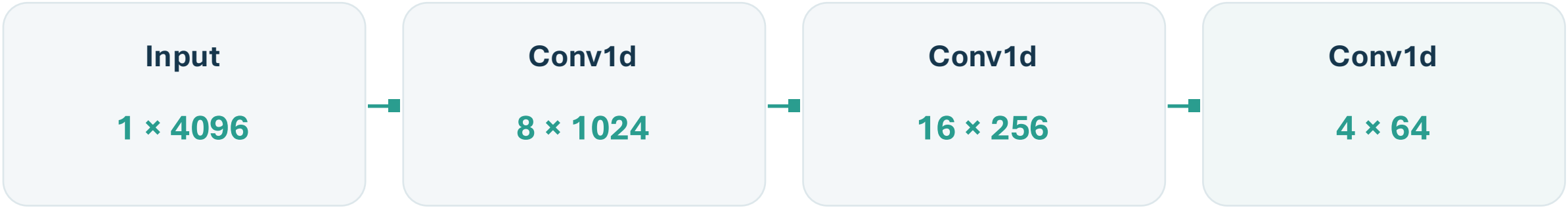
- Iteratively moves parameters in the direction that reduces the loss.
- This is the core optimization loop for training the encoder, quantizer support modules and decoder.

Learning rate

- Controls the step size of parameter updates.
- Too large can destabilize training; too small can make training very slow.

Neural codec quality depends on both architecture and training dynamics.

Example 2: a deeper convolutional network



Each stage uses stride 4. The decoder mirrors these stages with transposed convolutions.

4096 input samples → 256 latent floats (16× fewer values, not a 16× coded bitrate)

At 16 kHz: 4,096 samples per fragment · 3,253 trainable parameters

Open notebook: convolutional-autoencoder-complex.ipynb · Section 3

Train on fragments; test on new audio

Training

- Load one recording and split it into overlapping fragments.
- Learn encoder/decoder weights by minimizing a reconstruction loss.
- Compare waveform MSE and spectral loss.

Inference and evaluation

- Freeze the trained model.
- Encode–decode a different recording in blocks; use overlap-add.
- Listen and compare MSE/SNR on training vs unseen audio.

Good training reconstruction does not imply good generalization to other instruments or recordings.

Example: examples/convolutional-autoencoder-complex.ipynb · Lab 6

Example 2: train once, reconstruct new audio

TRAINING

- Chinese-orchestra sound (≈20 s)
- 154 overlapping frames of 4096 samples
- 80 epochs, waveform + spectral loss
- Update encoder and decoder parameters

INFERENCE

- Load independent trumpet recording
- Freeze learned parameters
- Encode and decode with 50% overlap
- Reassemble blocks using overlap-add

New audio can be processed with fixed weights; reconstruction quality may be poor on new material.

Open notebook: convolutional-autoencoder-complex.ipynb · Sections 2–6

Exmple 2: what does generalization mean?

Training recording

SNR	MSE
8.45 dB	0.00140

Unseen trumpet

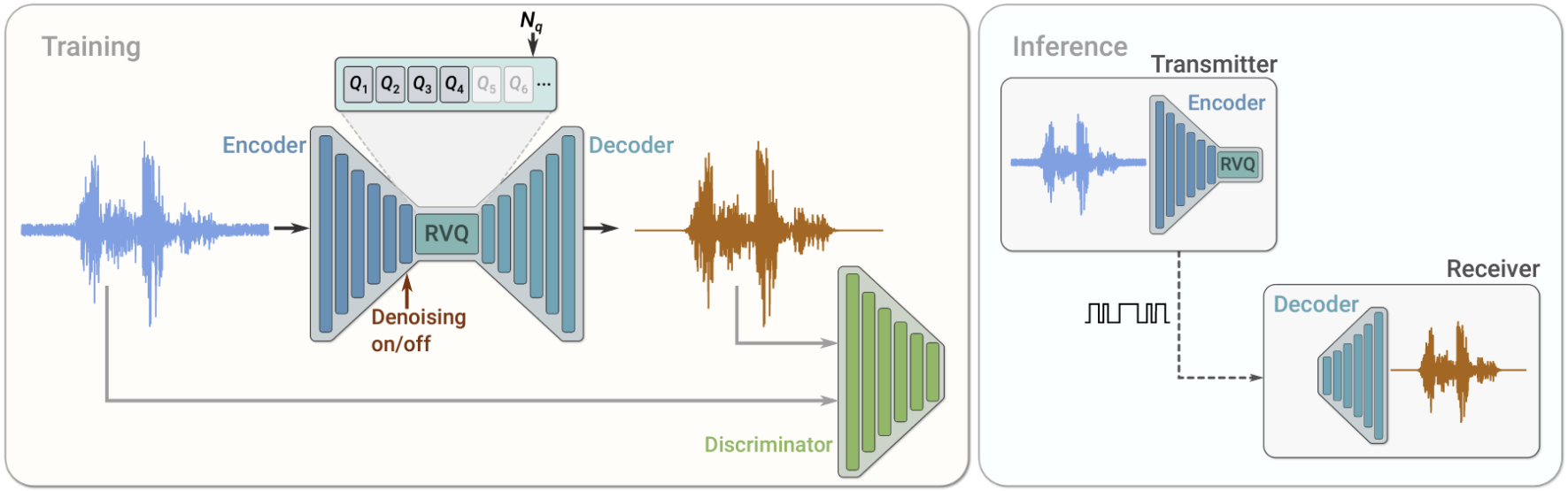
SNR	MSE
0.33 dB	0.01514

Discussion: the same network and latent size work better on training material than on an unfamiliar instrument.

Not a benchmark: these results depend on this one recording pair and training setup.

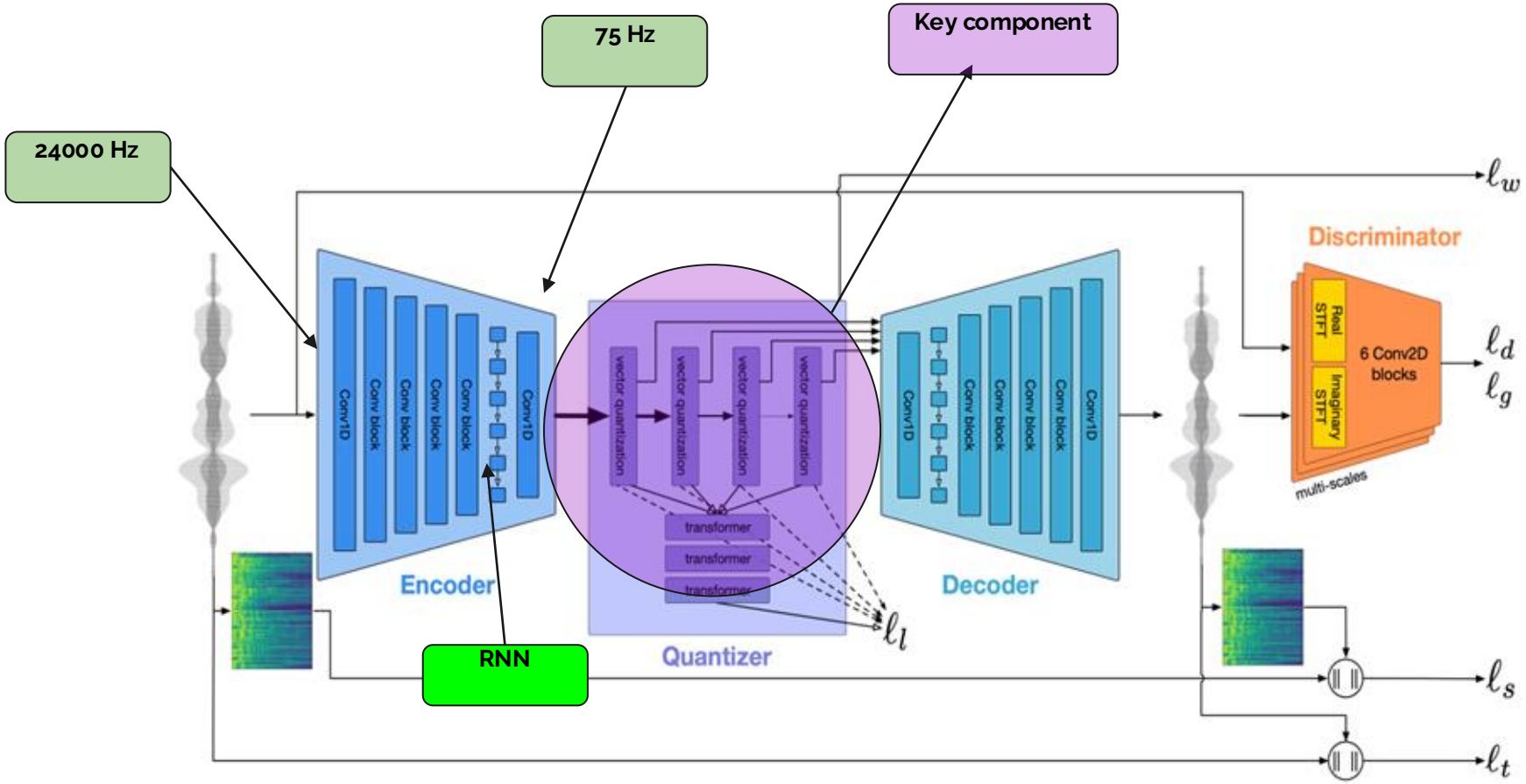
Source: convolutional-autoencoder-complex.ipynb · Quality comparison

SoundStream



Training combines reconstruction and adversarial losses; inference uses the encoder, RVQ codes and decoder.

EnCodec



Key idea: reduce a 24 kHz audio waveform to a low-rate discrete token stream, then reconstruct perceptually plausible audio.

Example: examples/encodec.ipynb

Reported compression results

Model	Bandwidth	Entropy Coded	Compression	MUSHRA
Reference	-	-	1×	**95.1**±1.8
MP3	64 kbps	-	24×	82.7±3.2
Opus	6 kbps	-	256×	17.7±5.9
Opus	24 kbps	-	64×	82.9±3.7
EnCodec	6 kbps	4.2 kbps	256×	82.9±2.4
EnCodec	12 kbps	8.9 kbps	128×	**88.0**±2.7
EnCodec	24 kbps	19.4 kbps	64×	**87.5**±2.6

MUSHRA scores compare perceived quality: the same nominal bitrate can sound very different depending on the model.

The discriminator and perceptual loss

- MSE measures waveform differences; spectral and perceptual objectives emphasize other aspects of reconstruction.
- A discriminator is trained to distinguish natural from generated audio.
- Adversarial training can improve plausibility without guaranteeing faithful reconstruction.

Risk: better perceived realism may not always mean faithful reconstruction.

Residual Vector Quantization (RVQ)

1. Map the input vector to the nearest code vector in a learned codebook.
2. Compute the residual between the input and the selected code vector.
3. Quantize the residual with another codebook.
4. Repeat across several layers to refine the approximation.

RVQ is the key quantization mechanism used by SoundStream and EnCodec.

Example: examples/encodec.ipynb

RVQ indices and nominal bitrate

Code tensor

Typical shape:

(batch, chunks, codebooks, token positions)

Each index selects one vector from its own codebook.

For K = 1,024 entries, each fixed-length index uses 10 bits.

Worked example

At F = 75 positions/s and Q = 32 codebooks:

$R = Q \times F \times \log_2(K) / 1000$

$R = 32 \times 75 \times 10 / 1000 = 24$ kb/s

Compare: 24 kHz / 16-bit mono PCM = 384 kb/s.

Nominal token payload excludes file framing, scale factors, packing overhead and padding.

Example: examples/encodec.ipynb

Optional transformer: modeling token context

- Self-attention weighs the relevance of other positions when processing one frame.
- This can model token statistics produced by the RVQ codebooks.
- Better token modeling can help entropy coding assign shorter bitstrings to common patterns.

Two sub-layers

Multi-head self-attention
Feed-forward network

Transformers are optional token models, not required to understand the convolutional encoder–decoder.

PyTorch implementation

- Open-source machine learning framework with a Pythonic interface.
- GPU acceleration is crucial for training large audio models.
- TorchAudio provides audio-specific operations such as spectrograms, mel-spectrograms and resampling.

A typical class demo can train a small convolutional autoencoder before moving to RVQ-based systems.

Challenges and current limitations

Compute and latency

Large architectures require compute power.

Low-latency real-time communication remains difficult.

Generalization

Models may struggle across speech, music and environmental audio.

Training needs large and diverse datasets.

Evaluation

Subjective tests such as MUSHRA are expensive and difficult to scale.

The best neural codec is not just the best model: it must be efficient, robust and audibly reliable.

Key takeaways

- Neural audio codecs learn analysis/synthesis transforms instead of relying only on fixed ones.
- CNN kernels extract local signal features; stride reduces time resolution.
- Training losses change reconstruction quality, even with the same latent size.
- A float-valued bottleneck is not yet a compressed bitstream: quantization and coding are needed.
- Generalization to unseen recordings is crucial, alongside compute, delay and listening quality.

References and notebook examples

Core papers

- Zeghidour et al. (2021). SoundStream: An End-to-End Neural Audio Codec.
- Défossez et al. (2022). High Fidelity Neural Audio Compression.

Course notebook examples

examples/convolutional-autoencoder-simple.ipynb

examples/convolutional-autoencoder-complex.ipynb

examples/encodec.ipynb

Course materials: github.com/xserra/audio-coding-materials/